

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to explain how a Hardware Security Module protects cryptographic key material and why it provides stronger assurance than software-based key storage.
- By the end of this lecture, you will be able to describe the difference between shared and dedicated HSM offerings in major cloud providers and identify the regulatory contexts that require dedicated options.
- By the end of this lecture, you will be able to explain how a Trusted Platform Module enables boot attestation and why remote attestation matters in cloud infrastructure you cannot physically inspect.
- By the end of this lecture, you will be able to define the secure-by-default principle and describe the operational and security consequences of shipping systems in an open-by-default state.
- By the end of this lecture, you will be able to apply management plane security principles to decisions about tool installation, access restriction, and package integrity verification.

Introduction

Every cloud environment has a security posture that was set, deliberately or by default, before the first application workload ever ran. The hardware components chosen for cryptographic operations, the firmware configuration of physical and virtual servers, and the settings that appear automatically when a new resource is provisioned all determine how defensible that environment will be under attack. Getting these foundational decisions right is not optional. It is the minimum required to avoid handing attackers the kind of persistent foothold that no amount of monitoring or response capability can fully compensate for.

This lecture covers three foundational building blocks of secure cloud infrastructure: Hardware Security Modules, Trusted Platform Modules, and the secure-by-default principle. Each represents a different layer of the infrastructure trust chain. HSMs anchor cryptographic key custody. TPMs anchor system boot integrity. Secure by default anchors the configuration baseline of every resource provisioned in the environment. Together, they define what it means to build infrastructure that you can trust before the first workload runs on it.

Alongside these hardware and configuration concepts, the lecture addresses management plane security. The management plane is the highest-privilege layer of any cloud environment. It controls what exists, how it is configured, and who can access it. Securing the management plane begins at the moment tools that interact with it are first installed, and it requires the same disciplined approach to access control, integrity verification, and minimisation that you would apply to the most sensitive application in your portfolio.

Core Concepts Explained

Hardware Security Modules (HSMs)

Definition: An HSM is a dedicated physical or virtual device that performs cryptographic operations and stores cryptographic key material in a tamper-evident hardware boundary. The key never leaves the device in plaintext form; all cryptographic operations are performed inside the module.

Business relevance: HSMs address the fundamental custody problem in key management: if key material is held in software memory, a sufficiently privileged attacker or a compromised process can retrieve it. HSMs eliminate that risk by making the key physically inaccessible even to administrators.

Cloud Security Practitioner relevance: A Cloud Security Practitioner selecting key management architecture must determine whether a shared HSM-backed managed service or a dedicated HSM is required based on regulatory obligations, audit requirements, and the sensitivity of the data being protected.

Real-world example: Financial institutions subject to Payment Card Industry Data Security Standard requirements need to demonstrate that cryptographic keys used for cardholder data encryption are managed in tamper-evident hardware that no other party, including the cloud provider, can access. A bank migrating its payment processing infrastructure to AWS faces a specific choice: AWS Key Management Service uses HSM-grade hardware on the backend but the physical devices are shared across multiple AWS customers. For PCI DSS scope where exclusive hardware control must be demonstrated to a Qualified Security Assessor, the bank instead provisions AWS CloudHSM, which allocates a dedicated physical HSM to the bank's account. The QSA can verify through AWS documentation that no other party can access the HSM, satisfying the requirement. The bank's cardholder data encryption keys are generated inside the CloudHSM, never exported in plaintext, and signing operations for TLS certificates and payment tokens happen within the device boundary. A penetration tester who compromises the application server still cannot retrieve the private keys because they never leave the HSM.

Risks or limitations: A common mistake is using HSMs for key storage but still allowing key export in plaintext for backup purposes, which defeats the purpose. The best practice is to use HSM-native key wrapping for backups: the backup key material is encrypted by another key that never leaves the HSM, so the exported blob is useless without the HSM.

Trusted Platform Modules (TPMs) and Boot Attestation

Definition: A TPM is a hardware chip, embedded in a motherboard or provided as a virtualised component, that provides a hardware root of trust by storing cryptographic keys and measuring the system boot process through Platform Configuration Register values.

Business relevance: In cloud environments where administrators cannot physically inspect servers, TPMs provide a hardware-rooted mechanism to verify that a system booted cleanly from a trusted configuration, without relying solely on software-level assertions that could be falsified.

Cloud Security Practitioner relevance: A Cloud Security Practitioner working with TPM-enabled infrastructure must understand how to use measured boot and remote attestation to verify host integrity before allowing sensitive workloads to run, and how to integrate PCR value comparison into automated provisioning workflows.

Real-world example: A healthcare cloud provider running workloads subject to HIPAA must ensure that the virtual machines processing patient data are running on hosts that have not been tampered with at the firmware or boot level. They configure their hypervisor hosts to use measured boot with vTPM components. When each host starts up, the vTPM records PCR measurements of the UEFI firmware, bootloader, and

hypervisor kernel. These measurements are compared against a signed baseline that was established when the host was initially validated and approved. If a host is restarted after a firmware update, the updated PCR measurements are compared against the new approved baseline. If a host's PCR values do not match any approved baseline, automated provisioning withholds workload assignment and flags the host for manual review. An attacker who installs a bootkit on the host or modifies firmware outside the approved change process will produce a PCR mismatch that the system detects before any patient data workloads are assigned. The healthcare provider can demonstrate to auditors that every host processing PHI has been attested as clean at every boot.

Risks or limitations: A widespread misconception is that TPMs primarily serve as storage devices for encryption keys. Their most operationally significant function is providing an integrity anchor through boot measurement. Organisations that deploy TPMs only for BitLocker or disk encryption but do not configure measured boot and remote attestation are leaving the most valuable capability unused.

Secure by Default

Definition: Secure by default means that a system, service, or resource is configured in its most secure state at initial provisioning. Unused ports are closed, unnecessary services disabled, default credentials absent or forced to change, logging enabled, and privileges minimal. Anything less secure must be explicitly chosen.

Business relevance: The alternative, open by default, reliably produces breached environments because hardening steps that require active effort are frequently skipped or incomplete. The cost of opening needed access is minor; the cost of a breach from never closing unnecessary access is major.

Cloud Security Practitioner relevance: A Cloud Security Practitioner enforces secure-by-default at the infrastructure-as-code level, in provisioning templates and cloud formation scripts, so that every resource that spins up inherits the secure baseline rather than requiring manual hardening after the fact.

Real-world example: The Mirai botnet, first identified in 2016, became one of the largest distributed denial of service infrastructures ever recorded by compromising hundreds of thousands of internet-connected cameras, routers, and other IoT devices. The attack method required no sophisticated exploitation: Mirai simply attempted to connect to devices using a list of factory default usernames and passwords, such as admin/admin and root/root, that the device manufacturers had never forced customers to change. The devices shipped open by default. Administrators never closed them. The botnet operators did not break any security controls; they walked through doors that were never locked. The practical lesson for cloud infrastructure is direct: when provisioning a new cloud database instance, public network access should default to disabled. When provisioning a new virtual machine, the root or administrator account should have no default password and should require a key or temporary credential immediately. When provisioning a new storage bucket, all public access should default to blocked. The administrator should have to take a deliberate, logged action to open each capability, ensuring that the risk decision is conscious and auditable.

Risks or limitations: Infrastructure-as-code templates that inherit vendor defaults without explicitly setting security parameters are a hidden source of open-by-default deployments. The best practice is to explicitly declare every security-relevant parameter in provisioning templates, even when the desired value matches the current default, so that future vendor default changes do not silently alter your security posture.

Management Plane Security

Definition: The management plane is the control layer of cloud infrastructure: the APIs, orchestration tools, and administration consoles through which all cloud resources are provisioned, configured, and monitored. It is the highest-privilege layer in any cloud environment.

Business relevance: Control of the management plane is equivalent to control of every resource beneath it. Attackers who achieve management plane access can create backdoor accounts, exfiltrate data, destroy infrastructure, or redirect traffic without ever touching individual workloads.

Cloud Security Practitioner relevance: A Cloud Security Practitioner is responsible for ensuring that management plane tools are installed from verified sources, access to management interfaces is restricted to defined roles and networks, and all management plane activity is logged and monitored for anomalous behaviour.

Real-world example: In the Capital One breach of 2019, a Server-Side Request Forgery vulnerability in a WAF allowed an attacker to reach the EC2 instance metadata service and retrieve an IAM role credential. That IAM role had excessive S3 permissions. The management plane, in this case the AWS IAM and S3 control interfaces, was the mechanism through which data was exfiltrated at scale. The attacker did not need to compromise the host operating system or break any encryption. Management plane credential access was sufficient. A best-practice posture would have restricted the IAM role to only the specific S3 buckets and operations it genuinely required, making the over-privileged credential far less valuable. For tool installation specifically: management tools such as Terraform, Ansible, AWS CLI, and kubectl should be installed only from verified official sources with cryptographic signature verification. They should run under dedicated service accounts with the minimum permissions required for each tool's specific function. Access to these tools on infrastructure hosts should be restricted by network controls and reviewed regularly.

Risks or limitations: Management plane tools installed with shared administrative credentials, without access logging, or from unverified sources represent one of the most common and consequential security gaps in cloud environments. The practice that prevents this is treating management tool installation as an access control event, not a routine software installation.

Practical Applications

The following scenarios illustrate how HSM, TPM, and secure-by-default principles apply in professional cloud security practice.

Designing Key Management Architecture for a Regulated Workload

A cloud security architect at a payments company is designing the key management architecture for a new card tokenisation service that must meet PCI DSS requirements. She evaluates three options: software-based key storage in application memory, cloud provider managed key management service backed by shared HSMs, and dedicated cloud HSM with customer-exclusive control. She documents the regulatory requirement: PCI DSS Requirement 3.7.1 requires that cryptographic keys used to protect cardholder data are protected against disclosure using dedicated hardware. She selects AWS CloudHSM and configures the service so that key generation happens inside the HSM at provisioning time, no key export operations are permitted in plaintext, and HSM audit logs are shipped to an immutable S3 bucket. She documents the architecture decision with the specific regulatory clause it satisfies, the key ceremony procedure for generating the initial key material inside the HSM, and the key rotation schedule. When the PCI QSA auditor reviews the tokenisation service, she provides the AWS documentation confirming single-tenant HSM allocation, the key ceremony record signed by two authorised witnesses, and the CloudHSM audit log demonstrating no plaintext key export events.

Implementing Secure-by-Default in an Infrastructure-as-Code Pipeline

A platform security engineer at a SaaS company discovers during a routine audit that six production S3 buckets were provisioned with public access enabled because the Terraform modules used by the development team

inherit AWS defaults rather than explicitly declaring security parameters. He updates the company's internal Terraform module library to explicitly set every security-relevant parameter: S3 public access block settings to true on all four attributes, default encryption to AES-256, bucket versioning to enabled, and server access logging to a designated audit bucket. He also adds a pre-deployment check using a policy-as-code tool that fails the pipeline if any S3 resource definition does not explicitly declare all four public access block parameters, regardless of their value. This ensures that future developers cannot accidentally inherit unsafe defaults by omitting parameters. He documents the change in the company's cloud security standard and presents it to the change advisory board. Within a month of the change, the automated check catches two additional cases where new team members had used third-party Terraform modules that did not set security defaults explicitly.

Responding to a Management Plane Credential Exposure

A cloud security engineer receives an alert from the cloud provider's threat detection service indicating that an IAM role used by a Terraform automation pipeline has made unusual API calls to services outside its normal operational scope: it queried the organisation's secrets manager service and attempted to list users in the directory service. The engineer identifies that the Terraform runner hosts pull credentials from an instance metadata service without requiring IMDSv2, which enforces session-oriented credential requests. She immediately revokes the specific IAM session token through the management console, rotates the IAM role's associated access key, and forces a new session for the automation pipeline. She then reviews CloudTrail logs for the previous 48 hours to identify all API calls made by the compromised credential, cross-referencing against the pipeline's known operational pattern. She finds no evidence of data exfiltration but documents all anomalous calls. She then updates the Terraform runner configuration to require IMDSv2, which prevents unauthenticated metadata requests, and reduces the IAM role to the minimum permissions required for Terraform's actual operational scope, eliminating the secrets manager and directory service permissions that made the anomalous calls possible.

Best Practices

- **Never allow plaintext export of cryptographic keys from HSMs.** Configure HSM key policies to prohibit plaintext export operations. Use HSM-native key wrapping for all backup procedures so that exported key material is encrypted by a wrapping key that itself never leaves the HSM. Document and test the key recovery procedure so that the organisation can recover from HSM failure without needing plaintext key access.
- **Configure measured boot and remote attestation, not just TPM presence.** A TPM that is present but not configured for boot attestation provides little operational security value. Set up PCR baseline measurements at initial host commissioning, store the approved baselines in a tamper-evident record, and integrate PCR comparison into automated provisioning so that hosts with mismatched measurements are withheld from workload assignment automatically.
- **Explicitly declare every security-relevant parameter in provisioning templates.** Do not rely on cloud provider defaults to be secure or stable. Every security-relevant setting, encryption, access controls, logging, network exposure, should be explicitly declared in infrastructure-as-code templates. This makes the security posture auditable, prevents silent changes when vendor defaults change, and enables policy-as-code tools to validate compliance before deployment.
- **Treat management tool installation as an access control event.** Before installing any tool that interacts with the cloud management plane, verify package integrity using the provider's published cryptographic signature. Assign the tool a dedicated service identity with the minimum permissions it requires. Log all

management plane tool installations in your CMDB. Review management tool access permissions quarterly and remove any permissions that have not been exercised in the review period.

- **Apply the principle of least privilege to HSM access and management plane roles in combination.** An IAM role that has both HSM administration permissions and the ability to create new IAM roles can effectively grant itself unlimited access to cryptographic material. Separate HSM administration duties from IAM administration duties, and require a multi-person review for any change to either role's permissions.

Common Mistakes and Misconceptions

Misconception: Deploying HSMs for key storage without disabling plaintext key export, which allows keys to be exfiltrated from the secure boundary.

Correct approach: Configure HSM key policies at creation time to prohibit plaintext export. Document and enforce key wrapping procedures for all backup and transport operations, and verify the policy is effective through periodic HSM audit log review.

Misconception: Treating TPMs as disk encryption accelerators rather than boot integrity anchors, missing the measured boot and remote attestation capabilities that provide meaningful cloud infrastructure assurance.

Correct approach: Configure TPM-enabled hosts for measured boot from initial commissioning, establish signed PCR baselines, and integrate attestation checks into automated provisioning workflows so that hosts with unexpected boot state changes are detected before workloads are assigned.

Misconception: Relying on cloud provider defaults for security settings in provisioning templates, creating invisible open-by-default configurations that persist until a breach or audit reveals them.

Correct approach: Explicitly declare all security-relevant parameters in every provisioning template, implement policy-as-code validation in the deployment pipeline, and audit deployed resources quarterly for drift from the declared baseline.

Misconception: Installing management plane tools under shared administrative credentials or without verifying package integrity, creating an uncontrolled high-privilege attack surface.

Correct approach: Assign dedicated service identities to management tools, verify cryptographic signatures before installation, log all management plane tool installations, and restrict network access to management interfaces to defined administrative paths only.

table: Cloud Infrastructure Trust Components: Selection and Configuration Guide

Use this reference table to evaluate which infrastructure trust component addresses each type of security requirement, what the key configuration decision is, and what the most common mistake to avoid is.

Component	Primary Security Function	Key Configuration Decision	Common Mistake	Best Practice
HSM (Shared)	Cryptographic key protection with HSM-grade hardware, shared physical device	Sufficient for most use cases; not suitable where exclusive hardware control must be demonstrated to auditors	Assuming shared HSM satisfies all PCI DSS or FIPS 140-2 Level 3 requirements	Verify regulatory requirement for dedicated vs shared before selecting service tier

HSM (Dedicated)	Cryptographic key protection with customer-exclusive physical device	Required when exclusive hardware control must be demonstrated; higher cost and management overhead	Not configuring key export restrictions, leaving plaintext export possible	Disable plaintext key export in HSM key policy at creation; use key wrapping for all backups
TPM / vTPM	Hardware root of trust; boot integrity measurement via PCR values	Enable measured boot and configure PCR baseline comparison; not just key storage	Using TPM only for disk encryption, leaving measured boot and attestation unconfigured	Establish signed PCR baselines at commissioning; integrate attestation into provisioning automation
Secure by Default	Configuration baseline; ensures resources start in secure state without manual hardening	Declare all security parameters explicitly in IaC templates; do not rely on vendor defaults	Inheriting vendor defaults that change over time or are insecure; omitting explicit declarations	Use policy-as-code to validate all security parameters are explicitly set before deployment
Management Plane Controls	Restrict and audit the highest-privilege layer of the cloud environment	Dedicated service identities, minimum permissions, cryptographic package verification, access logging	Installing management tools under shared admin credentials without integrity verification	Treat management tool installation as a change control event; log and review all management plane access

How to use this: This table represents a minimum review checklist. For each row, verify current state against the best practice column before any infrastructure provisioning event or security review.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A cloud security architect is designing key management for a payment processing system that must satisfy PCI DSS requirements. The requirement states that cryptographic keys protecting cardholder data must be held in tamper-evident hardware with no other tenant able to access the key material. Which solution best meets this requirement?

- A) A. AWS Key Management Service with customer-managed keys, because it uses HSM-grade hardware on the backend.
- B) B. A dedicated HSM service such as AWS CloudHSM that allocates exclusive physical hardware to the customer's account.
- C) C. Software-based key management with AES-256 encryption of the key files at rest.
- D) D. Storing keys in AWS Secrets Manager, which provides access control through IAM policies.

Answer: B. A dedicated HSM service such as AWS CloudHSM that allocates exclusive physical hardware to the customer's account.

Q2. A host in a cloud environment fails its boot attestation check. The Platform Configuration Register values do not match the approved baseline established at initial commissioning. What is the most appropriate immediate response?

- A) A. Reboot the host and run the attestation check again, as PCR values sometimes change transiently.

- B) B. Withhold workload assignment from the host and flag it for manual security review before returning it to service.
- C) C. Update the approved baseline to include the new PCR values so the host can return to service immediately.
- D) D. Re-image the host with the standard operating environment image and return it to the production pool.

Answer: B. Withhold workload assignment from the host and flag it for manual security review before returning it to service.

Q3. A development team's Terraform configuration provisions S3 buckets without explicitly setting public access block parameters, relying on AWS default settings. Which of the following represents the most significant risk of this approach?

- A) A. The buckets may be created in the wrong AWS region if defaults are applied.
- B) B. AWS may change its default settings for new bucket provisioning in the future, silently altering the security posture of newly created resources.
- C) C. Terraform will fail to provision the buckets because all parameters must be explicitly declared.
- D) D. The buckets will automatically be encrypted with AWS-managed keys, which may not satisfy compliance requirements.

Answer: B. AWS may change its default settings for new bucket provisioning in the future, silently altering the security posture of newly created resources.

Q4. An attacker compromises a build server that has the Terraform CLI installed and configured with production AWS credentials. The attacker uses Terraform to create a new IAM admin user with a known password. Which security principle was most directly violated to enable this attack?

- A) A. Encryption at rest, because the credentials should have been encrypted on the build server.
- B) B. Secure by default, because the build server should have blocked outbound connections to AWS by default.
- C) C. Least privilege and management plane security, because the Terraform credentials had permissions to create IAM users rather than only the permissions needed for Terraform's actual function.
- D) D. Network segmentation, because the build server should have been on a separate network from production.

Answer: C. Least privilege and management plane security, because the Terraform credentials had permissions to create IAM users rather than only the permissions needed for Terraform's actual function.

Q5. Which statement most accurately describes the primary security function of a Trusted Platform Module in a cloud environment?

- A) A. A TPM functions primarily as a hardware-accelerated key storage device that speeds up disk encryption operations.
- B) B. A TPM provides a hardware root of trust by recording and reporting measurements of the boot process, enabling verification that a host started from a known-good configuration.
- C) C. A TPM prevents virtual machines from reading each other's memory by enforcing hardware-level memory isolation.
- D) D. A TPM performs all cryptographic operations for the host, replacing software-based TLS and encryption libraries.

Answer: B. A TPM provides a hardware root of trust by recording and reporting measurements of the boot process, enabling verification that a host started from a known-good configuration.

Reflection Questions

1. Think about the most sensitive cryptographic material in an environment you are familiar with. Is that material currently held in software memory, a shared managed HSM service, or a dedicated HSM? Does the current approach match the regulatory and risk requirements for that data?
2. If a host in your infrastructure failed a boot attestation check tonight, how would your team find out? What is the current automated response, and is it fast enough to prevent sensitive workloads from running on a potentially compromised host?
3. Consider the last cloud resource your organisation provisioned. Were all security-relevant parameters explicitly declared in the provisioning template? Could you state with confidence what the security configuration of that resource is without reading the current state from the cloud provider console?
4. The Mirai botnet succeeded entirely because devices shipped with default credentials that were never changed. What defaults in your current environment have you verified rather than assumed are secure?
5. Management plane access is the most consequential access in a cloud environment. How many people, service accounts, and automated tools in your organisation currently have management plane credentials? When was the last time those permissions were reviewed against actual usage?

Key Takeaways

- HSMs protect cryptographic key material by ensuring the key never leaves the device in plaintext form; all operations are performed inside the hardware boundary.
- Shared and dedicated HSM services serve different regulatory contexts; PCI DSS and similar frameworks may require dedicated hardware to demonstrate exclusive custody.
- TPMs provide a hardware root of trust through boot attestation; their most operationally valuable function is measured boot and remote attestation, not key storage alone.
- Secure by default means security controls are active from initial provisioning; administrators must take a deliberate, logged action to reduce the security posture, not to increase it.
- The Mirai botnet of 2016 demonstrated that open-by-default configurations at scale create exploitable infrastructure without requiring any sophisticated attack technique.
- Management plane credentials represent the highest-privilege access in a cloud environment and must be scoped to minimum required permissions, verified for package integrity at installation, and subject to access logging.
- Provisioning templates should explicitly declare all security-relevant parameters rather than relying on vendor defaults, which can change and may not be secure in all contexts.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Identify one cryptographic key management decision in your environment or lab and document whether it uses software storage, a shared managed HSM, or a dedicated HSM, then verify that the choice matches the regulatory and risk requirements for the data it protects.
7. Configure a test virtual machine with a vTPM component and enable measured boot, then verify that a simulated boot modification produces a PCR mismatch that the attestation check catches.

8. Review one provisioning template in your environment and identify every security-relevant parameter that is not explicitly declared; add explicit declarations for each and test the updated template in a non-production environment.
9. Audit the IAM permissions of one management plane tool or service account in your environment against the actual API calls made in the past 30 days, and remove any permissions for services the tool has not legitimately called.